Lecture 11

Database Recovery

Dr. Vandana Kushwaha

Department of Computer Science
Institute of Science, BHU, Varanasi

Why Recovery Is Needed

- Whenever a **transaction** is submitted to a **DBMS** for **execution**, the **system** is **responsible** for **making sure** that :
  - **Either all** the **operations** in the transaction are **completed successfully** and their effect is **recorded permanently** in the **database**(**Transaction committed**).

    OR
  - That the **transaction** does **not have any adverse effect** on the **database** or any other transactions. (**Transaction aborted**)
- If a transaction **fails** after **executing** **some of its operations** but **before executing all of them**, the **operations already executed** must be **undone** to **ensure the Atomicity.**

Types of Failures

- There are **several possible reasons** for a **transaction to fail** in the **middle** of **execution:**
- **A Computer failure (system crash)**
  - A **hardware, software**, or **network error** occurs in the **computer system** during **transaction execution**.
- **A Transaction error**
  - **Transaction failure** may also occur because of **erroneous parameter values** or because of a **logical programming error.**
  - Some **operation** in the **transaction** may **cause** it to **fail**, such as **integer overflow** or **division by zero.**
  - Additionally, the **user may interrupt** the **transaction** during its **execution.**

Types of Failures

- **Local errors or exception conditions detected by the transaction**
  - During transaction execution, **certain conditions** may occur that necessitate **cancellation** of the **transaction.**
  - For example**, data** for the **transaction** may **not be found.**
  - An **exception condition**, such as **insufficient account balance** in a **banking database**, may cause a **transaction**, such as a **fund withdrawal**, to be **canceled.**
  - This **exception** could be **programmed** in the **transaction** itself, and in such a case **would not be considered** as a **transaction failure**.

Types of Failures

- **Failure due to Concurrency control enforcement**
  - The **concurrency control method** may decide to **abort a transaction** because it violates **serializability,** or it may **abort** one or more **transactions** to **resolve** a state of **deadlock** among several **transactions**.
  - **Transactions aborted** because of **serializability violations** or **deadlocks** are typically **restarted automatically** at a **later time.**
- **Disk failure**
  - Some **disk blocks** may **lose their data** because of a **read** or **write malfunction** or because of a **disk read/write head crash.**
  - This may **happen** during a **read** or **a write operation** of the **transaction.**

Types of Failures

- **Physical problems and Catastrophes**
- This refers to an **endless list** of **problems** that includes:
  - ***Power or Air-conditioning failure,***
  - ***Fire,***
  - ***Theft,***
  - ***Sabotage,***
  - ***Overwriting disks*** *or* ***tapes by mistake etc****.*

Recovery and Atomicity

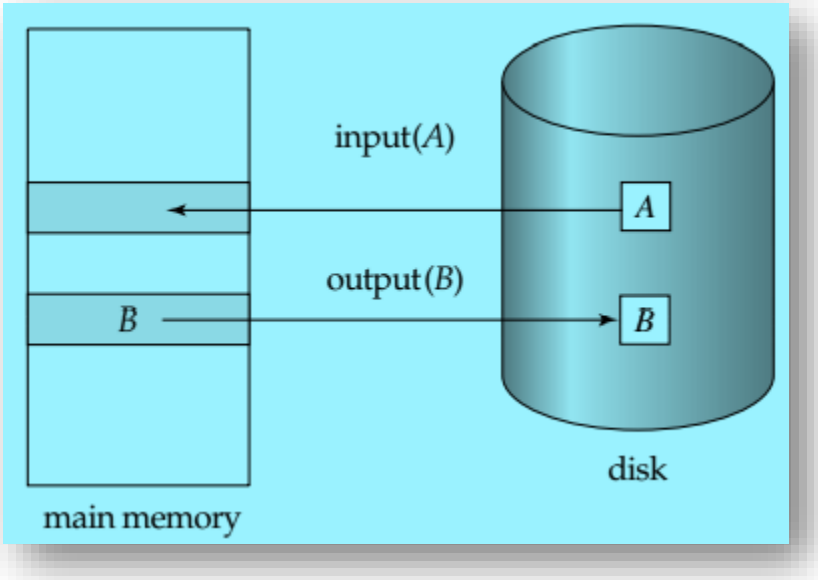
- Consider **transaction Ti** that transfers **$50** from **account A** to **account B.**
- **Atomicity goal** is either to perform **all** database modifications made by **Ti** or **none** at all.
- Multiple **read/write operations** may be required for **Ti** (to update **A** and **B**).
- A **failure** may occur after one of these **modifications** have been made but **before all of them** are made.
- To **ensure Atomicity despite failures**, we first **output(write) information** describing the **modifications to stable storage without modifying** the **database itself.**
- There are **two approaches** of **Database recovery: –**
  - ***Log-based Recovery***
  - ***Shadow-paging***
- We assume (initially) that **transactions** run **serially,** that is, one after the other.

Data Access

- The **Database system** resides **permanently** on **nonvolatile storage** (usually **disks**), and is **partitioned** into **fixed-length** storage **units** called **blocks.**
- **Blocks** are the **units** of **data transfer** to and from **disk**, and may contain several **data items.**
- **Transactions input(read)** information **from** the **disk** to **main memory**, and then **output(write)** the **information back onto** the **disk.**
- The **input** and **output operations** are done in **block units.**
- The **blocks** residing on the **disk** are **referred** to as **Physical blocks**;
- The **blocks residing temporarily** in **main memory** are referred to as **Buffer blocks**.

Data Access

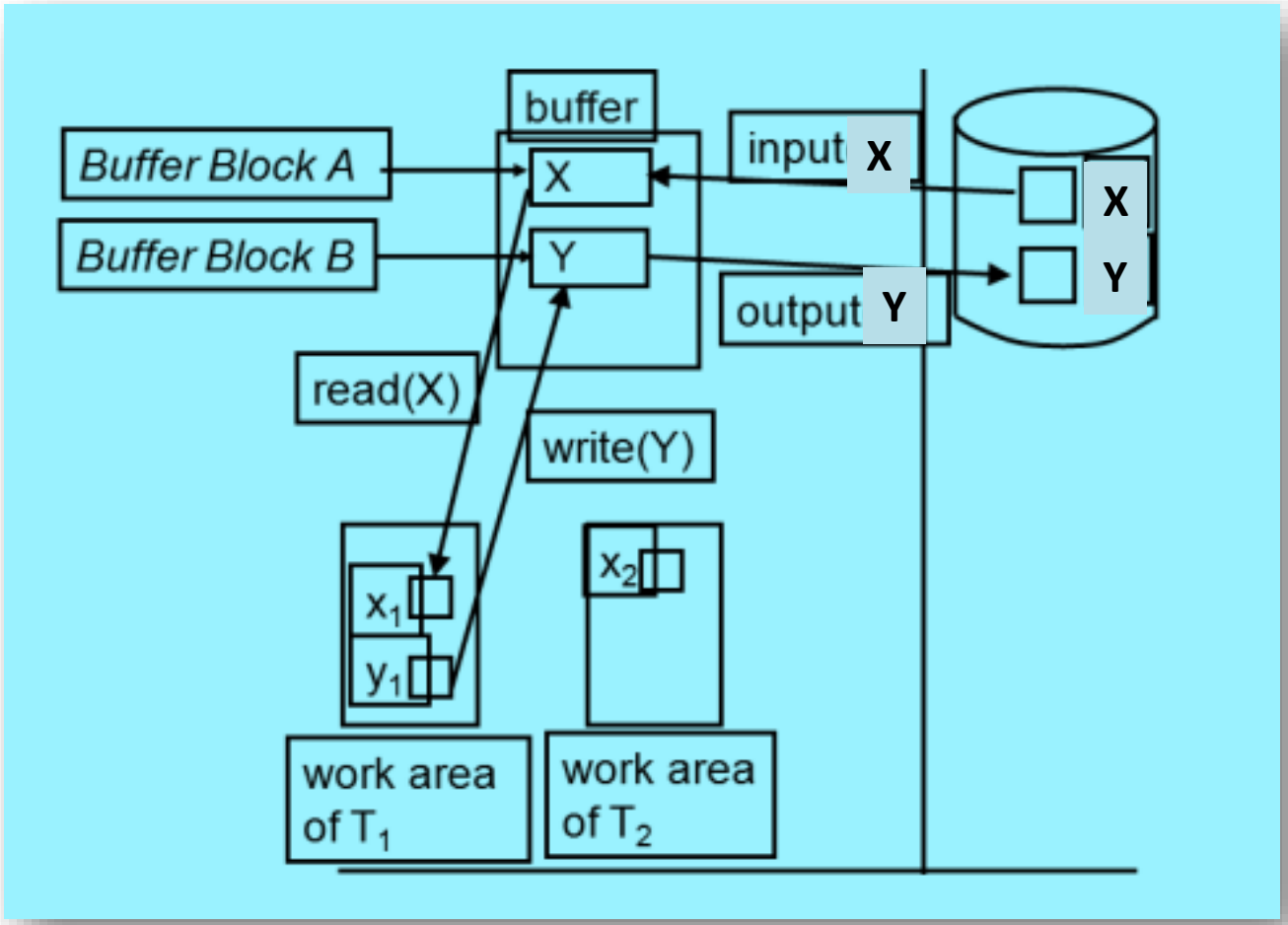
- **Block movements** between **disk** and **main memory** are **initiated** through the following **two operations:**
- **1. input(A)** transfers the **physical block A** to **main memory.**
- **2. output(B)** transfers the **buffer block B** to the **disk,** and **update** the appropriate **physical block** there.



Data Access

- Each **transaction Ti** has a **private work area**(in **main memory**) in which **copies** of **all the data items** accessed and updated by **Ti** are **kept.**
- The **system creates** this **work area** when the **transaction is initiated**;
- And the **system removes it** when the **transaction either commits or aborts.**
- Each **data item X** kept in the **work area** of **transaction Ti** is denoted by **Xi.**
- **Transaction Ti** interacts with the **database system** by **transferring data** to and from its **work area** to the **system buffer(main memory)** using **read** and **write operations.**

Data Access


buffer
Buffer Block A
Buffer Block B
X
Y
input
output
X
Y
X
Y
read(X)
write(Y)
x_1
y_1
x_2
work area of T_1
work area of T_2

Log-Based Recovery

- The most widely used structure for **recording database modifications** is the **log.**
- The **log** is a **sequence** of **log records**, recording all the **update activities** in the **database.**
- There are **several types** of **log records:**
  - **<Ti start>** Transaction Ti has **started.**
  - **<Ti, Xj, V1, V2>** Transaction **Ti** has performed a **write** on data item **Xj** . Xj had value **V1** before the write, and will have value **V2** after the write.
  - **<Ti commit>** Transaction **Ti** has **committed.**
  - **<Ti abort>** Transaction **Ti** has **aborted.**
- Whenever a **transaction performs a write,** it is **essential** that the **log record for that write** be **created before** the **database** is **modified.**
- For **log records** to be useful for **recovery** from **system** and **disk failures,** the **log must reside in stable storage.**

Log-Based Recovery

- There are **two techniques** using **Log-Based Recovery:**

  1. ***Deferred Database Modification***
  2. ***Immediate Database Modification***

Deferred Database Modification

- The **Deferred-modification technique** ensures **transaction atomicity** by **recording all database modifications** in the **log**, but **deferring** the **execution** of **all write operations** of a **transaction until** the **transaction partially commits**.
- A **transaction** is said to be **partially committed** once the **final action** of the **transaction** has been **executed.**
- If the **system crashes** **before** the **transaction completes** its **execution,** or if the **transaction aborts**, then **the information** on the **log** is **simply ignored.**
- Before **Ti starts** its execution, a record **<Ti start>** is written to the **log.**
- A **write(X)** operation by **Ti** results in the **writing** of a **new record <Ti, Xj, V1, V2>** to the **log.**
- Finally, when **Ti** **partially commits**, a record **<Ti commit>** is written to the **log.**
- When transaction **Ti** **partially commits**, the **records** associated with it in the **log** are **used** in **executing** the **deferred writes.**

Deferred Database Modification

- Since a **failure may occur** while this **updating** is taking place, **we must ensure** that, **before** the **start** of these **updates**, **all the log records** are **written out to stable storage.**
- Once **they have been written**, the **actual updating takes place**, and **the transaction enters** the **committed state.**
- Using the **log,** the **system** can handle **any failure** that results in the **loss of information** on **volatile storage.**
- The **recovery scheme** uses the following **recovery procedure**:
- **Redo(Ti)** - **sets** the **value** of **all data items updated by transaction Ti** to the **new values.**
- The **set of data items updated** by **Ti** and their **respective new values** can be **found in the log.**

Deferred Database Modification

- After a **failure,** the **recovery subsystem** consults the **log** to **determine** which **transactions need to be Redone.**
- **Transaction Ti needs** to be **redone if and only if** the **log contains both the record <Ti start>** and the record **<Ti commit>.**
- Thus, if the **system crashes** after the **transaction completes its execution (partially committed)**, the **recovery scheme uses the information** in the **log to restore the system** to a **previous consistent state** after the transaction had **completed.**

Deferred Database Modification

- Let **T_0** be a transaction that transfers **$50** from account **A** to account **B**:
- Let **T_1** be a transaction that withdraws **$100** from account **C.**
- Suppose that these transactions are **executed serially**, in the order **T_0** followed by **T_1.**
- Let the values of accounts **A, B**, and **C** before the execution took place were **$1000, $2000,** and **$700,** respectively.
- The portion of the **log** containing the relevant information on these **two transactions**.

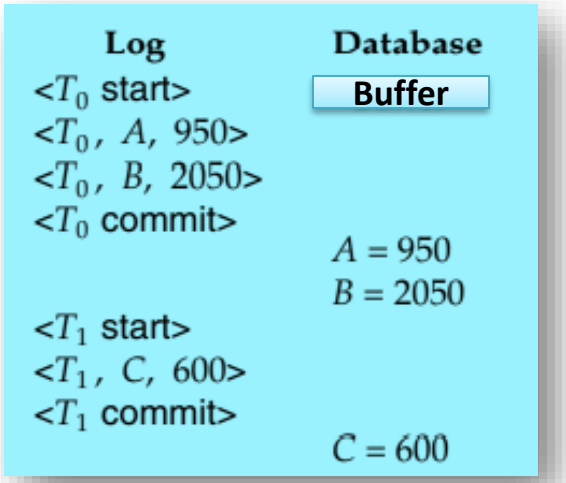
T_0: read(A);
$A := A - 50$;
write(A);
read(B);
$B := B + 50$;
write(B).

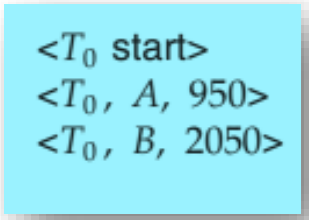
T_1: read(C);
$C := C - 100$;
write(C).

<T_0 start>
<T_0, A, 950>
<T_0, B, 2050>
<T_0 commit>
<T_1 start>
<T_1, C, 600>
<T_1 commit>

Deferred Database Modification

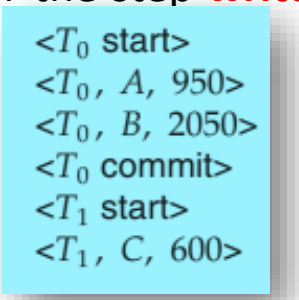
- State of the **log** and **database** corresponding to T_0 and T_1.
- **CASE 1:**
- Assume that the crash occurs just after the log record **<T_0, B, 2050>** for the step **write(B)** of transaction T_0 has been written to **stable storage.**
- The **log** at the time of the **crash** appears in **Figure** :
- When the **system** comes back up, **no redo** actions need to be taken, since **no commit** record appears in the **log.**
- The values of accounts **A** and **B** remain **$1000** and **$2000**, respectively.
- The **log records** of the **incomplete** **transaction T_0 can be deleted** from the **log.**





Deferred Database Modification

- **CASE 2:**
- Now, let us assume the **crash** comes **just after** the **log record** for the step **write(C)** of transaction T_1 has been written to **stable storage**.
- In this case, the **log** at the ***time of the crash*** is as in **Figure**.



- When the system comes back up, the **operation redo(T_0) is performed**, since the records **<T_0 start>** and **<T_0 commit>** both appears in the **log.**
- After this operation is executed, the values of accounts **A** and **B** are **$950** and **$2050**, respectively.
- The value of account **C** remains **$700.**
- As before, the **log records** of the **incomplete transaction T_1** can be **deleted** from the **log**.

Deferred Database Modification

- **CASE 3:**
- Finally, assume that a **crash** occurs just after the **log record** is written to **stable storage.**
- The **log** at the time of this **crash** is as in Figure.

<T_0 start>
<T_0, A, 950>
<T_0, B, 2050>
<T_0 commit>
<T_1 start>
<T_1, C, 600>
<T_1 commit>

- When the system comes back up, **two commit records** are in the **log**: one for T_0 and one for T_1.
- Therefore, the system must perform operations **redo(T_0)** and **redo(T_1),** in the order in which their **commit** records **appear** in the **log.**
- After the **system** executes these operations, the **values** of accounts **A, B,** and **C** are **$950, $2050**, and **$600,** respectively.

Deferred Database Modification

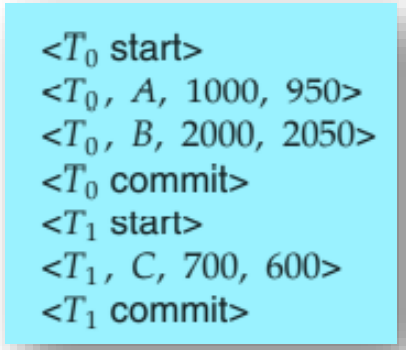
- The **REDO** is needed in case the **system fails after** the **transaction partially commits** but **before** all **its changes** are **recorded** in the **database on Hard disk**.
- In this case, the **transaction operations** are **redone** from the **log entries.**

Immediate Database Modification

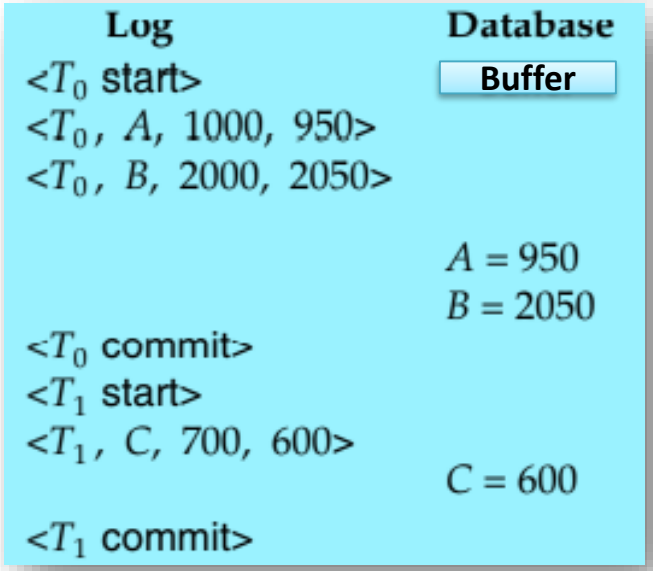
- The **Immediate-modification technique** allows **database modifications** to be **output** to the **database** while the **transaction is still** in the **active state.**
- **Data modifications** written by **active transactions** are called **uncommitted modifications.**
- In the event of a **crash** or a **transaction failure**, the **system** must **use the old-value field of the log records** to **restore the modified data items** to the value they had prior to the start of the transaction.
- The **Undo operation** accomplishes this **restoration.**
- Before a **transaction Ti starts** its execution, the **system** writes the record **<Ti start>** to the **log.**
- During its execution, any **write(X)** operation by **Ti** is preceded by the writing of the appropriate new update record to the **log.**
- When **Ti partially commits**, the **system writes** the **record <Ti commit>** to the **log.**

Immediate Database Modification

- Before execution of an **output(B) operation**, the **log records corresponding to B** be **written onto stable storage.**
- Consider the transactions T_0 and T_1 and the portion of the **system log** corresponding to T_0 and T_1:



- The state of **system log** and **database** corresponding to T_0 and T_1 is:



Immediate Database Modification

- Using the **log**, the system can **handle** any **failure** that does not result in the **loss** of information in **nonvolatile storage.**
- The recovery scheme uses **two recovery procedures:**
  - **Undo(Ti)** restores the value of all data items updated by transaction **Ti** to the **old values.**
  - **Redo(Ti)** sets the value of all data items updated by transaction **Ti** to the **new values.**
  - The set of **data items** updated by **Ti** and their respective **old** and **new values** can be **found** in the **log.**

Immediate Database Modification

- After a **failure** has occurred, the **recovery scheme** consults the **log** to determine which **transactions** need to be **redone**, and which need to be **undone:**
  - **Transaction Ti** needs to be **Undone** if the log contains the record **<Ti start>** , but does **not contain** the record **<Ti commit>.**
  - **Transaction Ti** needs to be **Redone** if the log contains **both** the record **<Ti start>** and **<Ti commit>** .

Immediate Database Modification

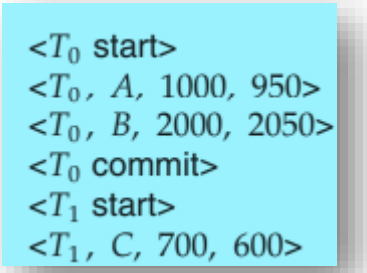
- Suppose that the **system crashes** before the completion of the transactions.
- **Case 1:** the **crash occurs** just after the **log record** for the step **write(B)** of transaction T_0 has been written to **stable storage**.

<T_0 start>
<T_0, A, 1000, 950>
<T_0, B, 2000, 2050>

- When the system comes back up, it finds the record **<T_0 start>** in the **log**, but **no** corresponding **<T_0 commit>** record.
- Thus, transaction T_0 must be **undone,** so an **Undo(T0)** is performed.
- As a result, the values in accounts **A** and **B** (on the disk) are **restored** to **$1000** and **$2000**, respectively.

Immediate Database Modification

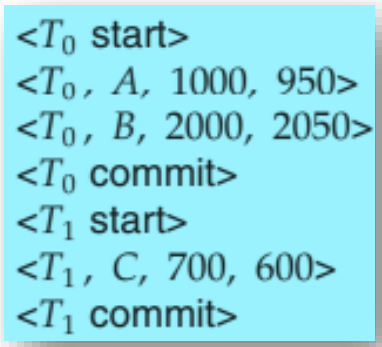
- **Case 2:** Next, let us assume that the crash comes just after the **log record** for the step **write(C)** of transaction **T1** has been written to **stable storage**.



- When the system comes back up, **two recovery actions** need to be taken.
- The operation **Undo(T1) must be performed**, since the record **<T_1 start>** appears in the log, but there is no **<T_1 commit>** record .
- The operation **Redo(T_0) must be performed**, since the log contains both the record **<T_0 start>** and the record **<T_0 commit>** .
- At the end of the entire recovery procedure, the values of accounts **A, B**, and **C** are **$950, $2050,** and **$700**, respectively.

Immediate Database Modification

- **Case 3:** let us assume that the crash occurs just after the log record **<T_1 commit>** has been written to stable storage.



- When the system comes back up, **both T0 and T1 need to be redone.**
- Since the records **<T0 start>**and **<T0 commit>** appear in the **log**, as do the records **<T1 start>** and **<T1 commit>** .
- After the system performs the recovery procedures **redo(T0)** and **redo(T1),** the values in accounts **A, B**, and **C** are **$950, $2050**, and **$600**, respectively.

Immediate Database Modification

- Notice that during **Immediate Database Modification** it is **not a requirement** that **every update** be **applied immediately** to **disk**; it is just possible that **some updates** are **applied** to **disk** before the **transaction commits**.
- Theoretically, we can distinguish **two main categories** of **Immediate update algorithms.**
  - If the **recovery technique** ensures that **all updates** of a **transaction** are **recorded** in the **database** on **disk** ***before*** *the* ***transaction commits,*** there is **never** a **need** to **REDO** any **operations** of **committed transactions.**
  - This is called the **UNDO/NO-REDO Recovery algorithm**.

Immediate Database Modification

- On the other hand, if the **recovery technique** found that **only few updates** of a **transaction** are **recorded** in the **database** on **disk *before the transaction commits***, there is a **need** to **REDO** **all the operations** of **committed transactions**.
- This is called the **UNDO/REDO** **recovery algorithm**.

Checkpoints

- In **Log-based Recovery techniques** when a **system failure** occurs, we must **consult** the **log** to **determine** those **transactions** that **need** to be **redone** and those that **need** to be **undone.**
- We need to **search** the **entire log** to **determine** this **information** which is **time consuming.**
- To **reduce** these types of **overhead**, **Checkpoints** were **proposed.**
- During **Transaction execution**, the **system maintains** the **log,** using one of the **two techniques** described earlier.

Checkpoints

- The **system periodically performs Checkpoints**, which require the following **sequence of actions** to take place:
  - **1. Output** onto **stable storage** all **log records** currently residing in **main memory.**
  - **2. Output** to the **disk all modified buffer blocks.**
  - **3. Output** onto **stable storage** a log record **<checkpoint>.**

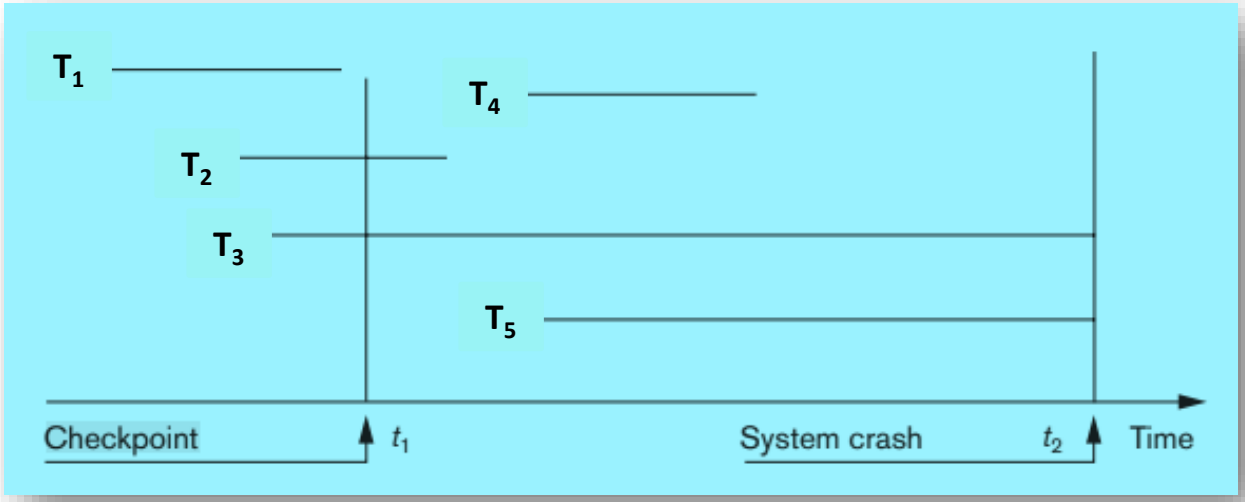
Checkpoints

- **Transactions** are **not allowed** to perform **any update actions**, such as **writing** to a **buffer block** or writing a **log record**, while a **checkpoint is in progress.**
- Consider a **transaction Ti** that **committed prior** to the **checkpoint.**
- For such a transaction, the **<Ti commit>** record appears in the **log** before the record **<checkpoint>.**
- Any **database modifications** made by **Ti** must have been **written** to the **database** either **prior to the checkpoint** or as **part** of the **checkpoint** itself.
- Thus, at **recovery time**, there is **no need to perform** a **Redo operation on Ti.**

Checkpoints

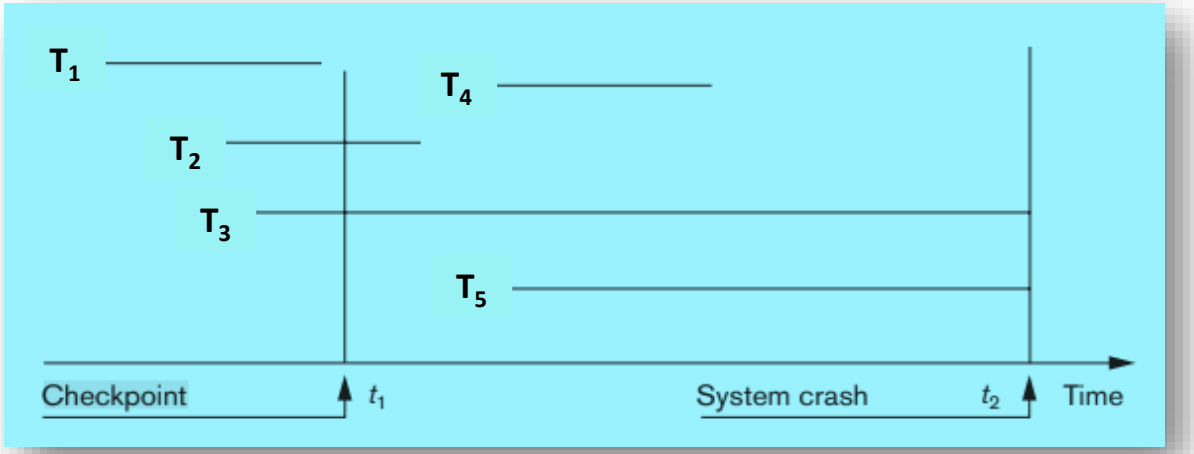
- After a **failure has occurred**, the **recovery scheme** examines the **log to determine the** transaction **Ti** that needs to be **Redone** and transaction **Ti** that needs to be **Undone.**
- **Recovery scheme** can find such a **transaction** by **searching** the **log backward**, from the **end** of the **log**, until it finds the **last <checkpoint> record**.
  - If the **log** contains operations like **<T_i, start>** and **<T_i, commit>** both after the last **checkpoint the T_i** needs to be **Redone.**
  - If the **log** contains operations like **<T_i, commit> alone** after the last **checkpoint the T_i** needs to be **Redone.**
  - If the **log** contains operations like **<T_i, start>** but no **<T_i, commit>** after the last **checkpoint** then **T_i** needs to be **Undone.**

Example : Checkpoints



- When the **checkpoint** was taken at **time t1**, **transaction T_1** had **committed,** whereas **transactions T_2 and T_3** had **not.**
- Before the **system crash** at **time t2**, **T_2 and T_4** were **committed** but **not T_3 and T_5.**
- There is **no need to Redo transaction T_1**—or any **transactions committed** before the **last checkpoint time t1.**

Example : Checkpoints



- The **transactions T_2 and T_4 must be Redone**, however, because **both transactions** reached their **commit points after** the **last checkpoint.**
- **Transactions T_3 and T_5 must be Undone :** because their **write operations** were **recorded** in the **database** on disk under the **Immediate update protocol.**
- **Transactions T_3 and T_5 are ignored:** They are effectively **canceled** or **rolled back** because none of their **write operations** were **recorded** in the **database** on **disk** under the **Deferred update protocol.**

Shadow Paging Scheme

- **Shadow paging** is an **alternative** to **log-based recovery**; this scheme is useful if **transactions** execute **serially.**
- **Shadow paging** considers the **database** to be **made up** of a number of **fixed size disk pages** (or **disk blocks**)—say, **n**—for recovery purposes.
- A **directory (page table)** with **n entries** is constructed, where the **i^{th} entry** points to the **i^{th} database page** on **disk.**
- When a **transaction** begins executing, the **current directory (page table)** —whose entries point to the most recent or **current database pages** on **disk**—is copied into a **shadow directory (page table)**.
- The **shadow directory (page table)** is then **saved on disk** while the **current directory** is **used** by the **transaction.**

Shadow Paging Scheme

- **Transaction Execution**
- During **transaction execution**, the **shadow directory (page table)** stored on **disk** is **never modified.**
- When a **write operation** is performed, a **new copy** of the **modified database page** is **created**, but the **old copy** of that **page** is **not overwritten.**
- Instead, the **new page** is **written elsewhere**—on some previously **unused disk block**.
- The **current directory (page table)** entry is **modified** to **point** to the **new disk block**, whereas the **shadow directory (page table)** is **not modified** and continues to point to the **old unmodified disk block**.

Shadow Paging Scheme

- **Transaction Commit:**
- All the **modifications** which are done by **transaction** which are present in **main memory buffers** are **transferred** to **physical database**.
- Output **Current page table** to **disk.**
- Make the **Current page table** the **New Shadow page table**, as follows:
  - Keep a **pointer** to the **Shadow page table** at a **fixed (known) location** on **disk.**
  - To make the **Current page table** the **new Shadow page table**, simply **update the pointer** to point to **Current page table** on **disk.**
- Thus If the **transaction** is **successful,** the **Shadow page table** is **replaced** by the **Current page table.**

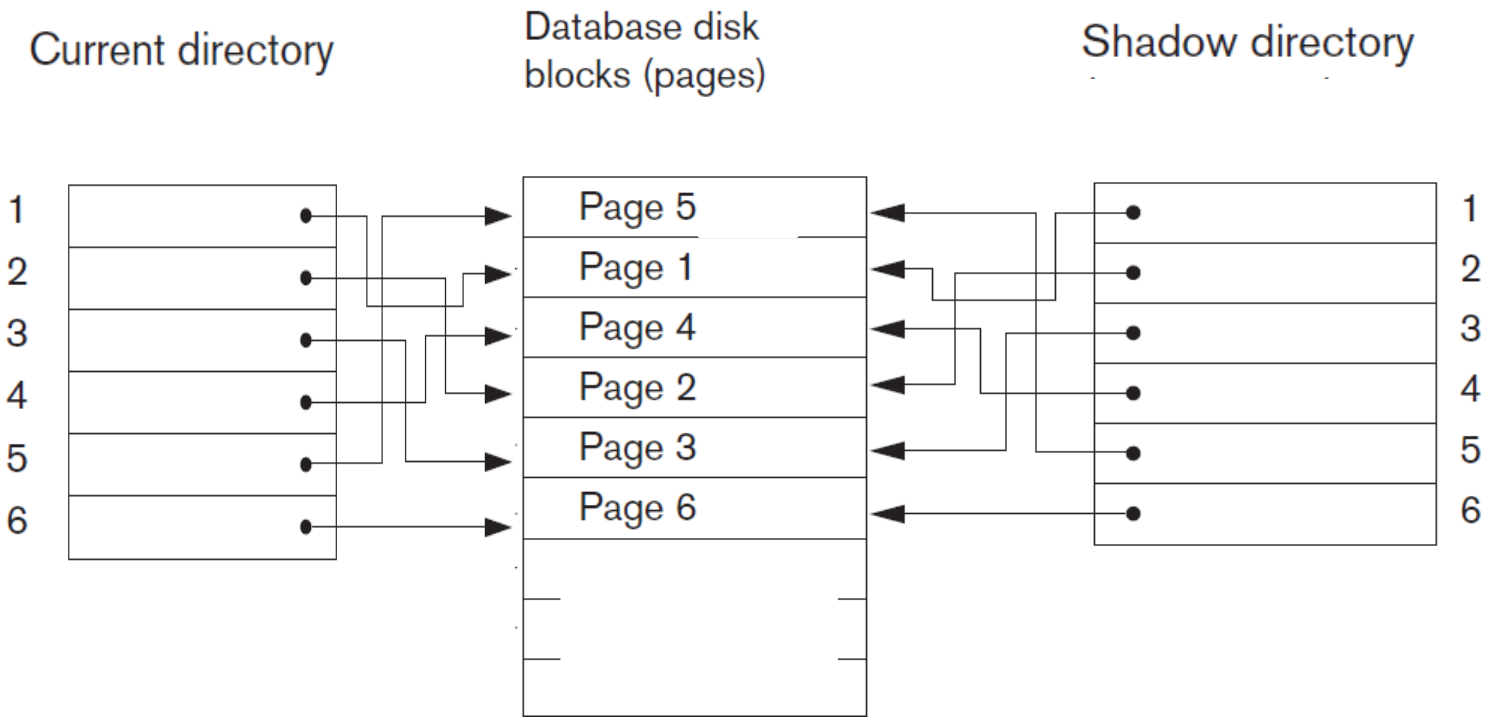
Shadow Paging Scheme

- **Transaction Abort:**
- If the **transaction** is **aborted**, the **Current page table** is **discarded**, leaving the **Shadow page table intact.**
- Since the **Shadow page table** still points to the **original pages**, **no changes** are **reflected** in the **database.**

Shadow Paging Scheme: Example

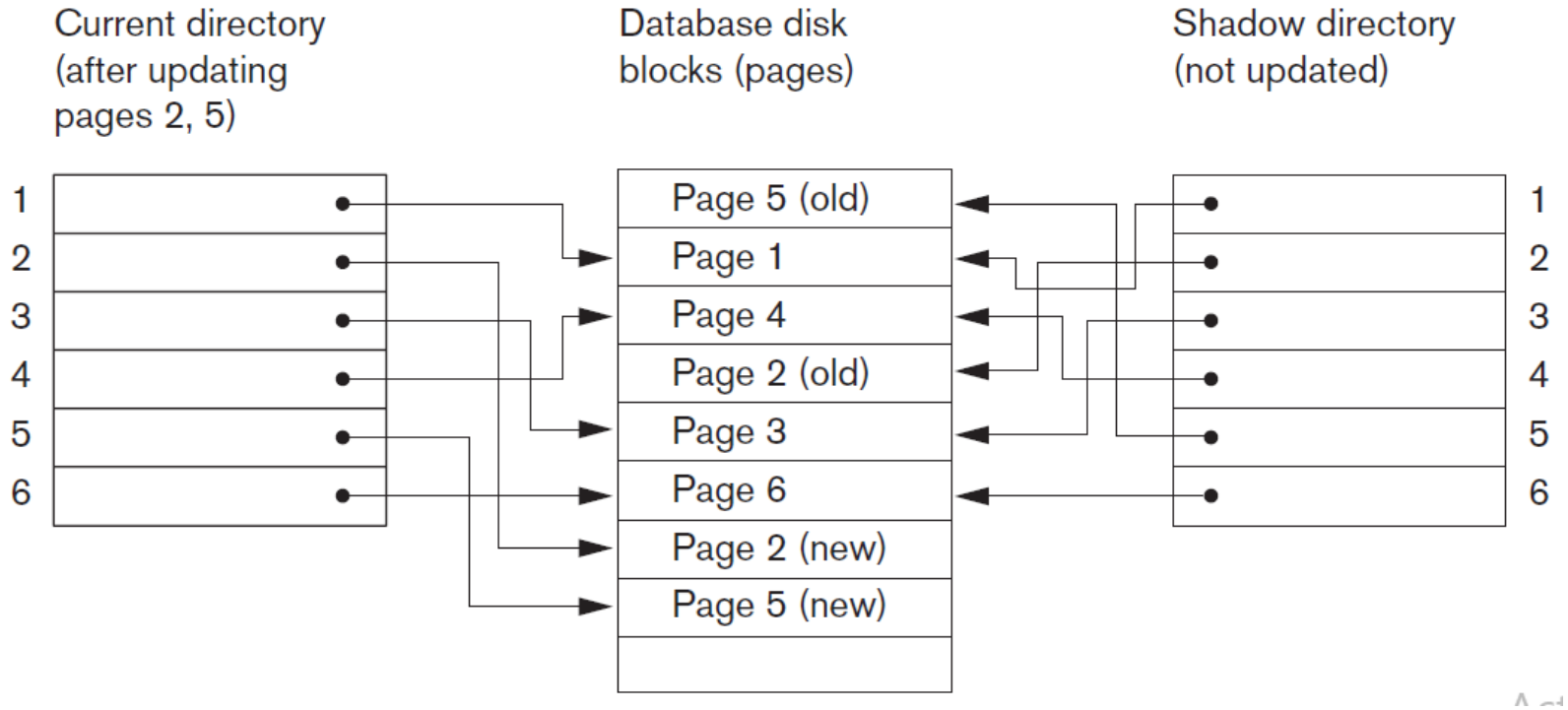
- Let **two write operations** are performed by a **transaction** on **page 2** and **page 5.**
- **Before** start of **write operation** on **page 2**, **current page table** points to **old page 2**. When **write operation starts** following **steps** are performed :
  - Search for **available free block** in **disk blocks.**
  - After finding **free block**, it copies **modified page 2** to **free block** which is represented by **Page 2 (New).**
  - Now **Current page table** points to **Page 2 (New)** on **disk** but **Shadow page table** points to **old page 2** because it is **not modified.**
  - The changes are now propagated to **Page 2 (New)** which is pointed by **Current page table.**

Shadow Paging Scheme



Page Tables before the **Transaction Execution**

Shadow Paging Scheme



Page Tables after the Transaction Execution

Shadow Paging Scheme

- **Advantages**
- **No overhead** of writing **log records**.
- **Database Recovery** is **quick** and **inexpensive**, with **no need** for **Undo** or **Redo** operations.
- **Disadvantages**
- **Commit overhead** is **high** as it need to **flush** every **updated page**, and **page table**.
- Data gets **fragmented** (related pages get separated on disk)
- After each **transaction**, **pages** containing **old versions** of **modified data** must be **garbage collected** and **added** to the **list** of **unused pages.**
- **Hard** to **extend algorithm** to allow **transactions** to run **concurrently.**